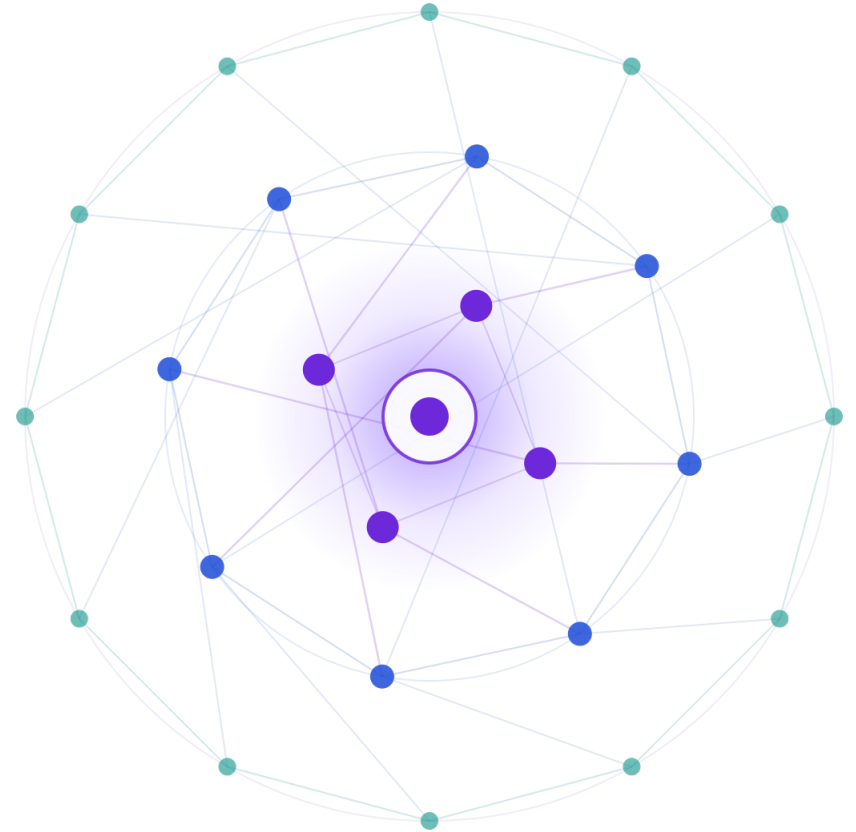


Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID -
- Team Name - MSRIT_MindMesh
- College Name - MSRIT
- Member Name & Email 1- Pratheek Ravi 1ms24cs130@msrit.edu
- Member Name & Email 2- Pranav Kumar 1ms24cs127@msrit.edu
- Member Name & Email 3- S Vaishnavi 1ms24cs150@msrit.edu
- Member Name & Email 4- Ritisha Purkait 1ms24ee048@msrit.edu
- Submission Github link - <https://github.com/jinxrift/SamsungSmartTroubleShooter2026.git>



Theme

- **Theme 2 — Troubleshooting** Smart Guided Troubleshooting Engine
- **Core idea:** transform natural-language Galaxy device complaints into clean, validated, one-tap troubleshooting plans.
- **Target state:** screen-specific actions, correct action ordering, and verified in-app Settings deeplinks.
- **Design target:** return previously resolved cached scenarios in under 300 ms.

Existing Solutions & Gaps

- **Current state:** support agents interpret unstructured knowledge-base/SIIS content, identify causes, collect relevant steps, and order them manually.
- **User gap:** complaints such as “screen flickers” or “swipes go the wrong way” do not directly identify the required device setting.
- **Action gap:** text instructions still require users to navigate nested Settings menus themselves.
- **Operational gap:** manual triage can take roughly 15 minutes per scenario.
- **Technical risks:** cache fragmentation, LLM hallucination, weak parent-menu matching, URL injection, and overly granular or duplicate actions.

Our Solutions & Architecture Diagram

Raw complaint + optional SIIIS reference

- → 0 Query Enrichment
Normalize + paraphrase
- → 1 Structure Extraction
Goal / Actions / Steps
- → 2 Deeplink Mapping
Resolve + sequence
- → 3 Fast-Path Cache
Reuse verified plans
- → 4 REST API
Validated JSON response
- **Output: actionable deeplink + validation deeplink + schema/operational metadata**

Demo & Product Walkthrough

- **Example input:** “The mobile phone swipe navigation moves up or down instead of left or right after downloading an app.”
- **Extracted goal:** Follow these steps to perform this Swipe Navigation Troubleshooting.
- **Action:** Configure Navigation Bar Settings.
- **Actionable path:** Settings → Display → Navigation bar → choose Buttons or Swipe gestures.
- **Example response metadata:** latency_ms = 212; cache_hit = true; model = gpt-4o-mini; cost_usd = 0.0.
- **Fallback behavior:** a reserved bixby://dummy_positive placeholder is used only when a valid Settings screen is not currently indexed.

Tools and tech stack used

- **Backend/API:** Python + FastAPI; REST endpoints /v1/troubleshoot and /health.
- **Validation:** Pydantic models and strict schema constraints for Goal, Action, StepGroup and Deeplink objects.
- **AI layer:** OpenAI-compatible LLM integration for optional grounded extraction.
- **Retrieval:** deeplinks catalog with semantic and keyword search; roadmap specifies BM25 + dense embeddings.
- **Caching:** persistent/local semantic cache with pre-computed query variations for fast-path responses.
- **Data assets:** queries.json, siis_responses.json, deeplinks.json, samples/, schema.py.

Impact & Use case

- **Primary impact:** bridge the gap between support knowledge and actual device actions.
- **For users:** clearer troubleshooting plans with screen-specific actions and one-tap Settings entry points.
- **For support teams:** less manual interpretation and more consistent action ordering.
- **Supported domains:** Battery, Display, Camera and Performance.
- **System use cases:** customer-support assistants, smart troubleshooting workflows and REST-based support integrations.
- **Performance goal:** cached scenarios target P95 latency ≤ 300 ms.

Innovation highlights, results and limitations

- **Innovation:** combines grounded structure extraction, catalog-based deeplink resolution, semantic caching and deterministic validation.
- **Safety/quality:** zero web URLs, no invented deeplinks, no hallucinated solutions, and strict source adherence.
- **Evaluation targets:** schema validity $\geq 99\%$; rule compliance $\geq 95\%$; exact deeplink catalog validity 100%; valid actionable deeplink rate $\geq 90\%$; semantic paraphrase hit rate $\geq 80\%$.
- **Latency targets:** cache-hit paths ≤ 300 ms P95; cold full-pipeline generation ≤ 8000 ms P95.
- **Observed example:** the worked API response records a 212 ms cache hit.
- **Limitations:** multi-intent cases, domain gaps, settings-hierarchy variations, masked-URI matching and prompt-only constraints remain edge cases.

What's next

- **Phase 1 — Foundation & Validation:** schema harness, programmatic validators and baseline parsing tests.
- **Phase 2 — Semantic Indexing & Deeplink Mapping:** BM25 + dense retrieval, screen resolution and action ordering.
- **Phase 3 — Fast-Path Caching:** semantic normalization, intent extraction, persistent local cache and latency benchmarks.
- **Phase 4 — API Assembly & Verification:** REST endpoints, graceful fallbacks and end-to-end stress tests.
- **Future expansion:** broader device categories, more natural-language variations, support dashboards and analytics.

Brownie points slide(differentiation)

- **Catalog integrity first:** deeplinks are matched from metadata rather than generated or altered by the model.
- **No-hallucination path:** plans are grounded in supplied SIIS/reference text; no viable solution can return an empty context with a no-match fallback.
- **Hierarchy-aware sequencing:** safe toggles and non-invasive steps come before critical or disruptive operations.
- **Deterministic fallback:** programmatic validation and fallback logic continue to work when AI extraction is unavailable or invalid.
- **Semantic cache:** paraphrased user queries can reuse verified plans instead of requiring a fresh model call.
- **Operationally measurable:** latency, cache-hit state and cost metadata are exposed with the API response.

Checklist- Updated on Public GitHub

- Working prototype code — public or shared GitHub repo (Y)
- README with reproducible setup instructions (Y)
- Demo video, max 5 minutes (YouTube or Drive link)
- Presentation file (PPT or PDF) (Y)

Thank you

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India